Chapter 26 worked solutions

Models for Code

These solutions cover Exercises 1–4, which use the interval contract and the chapter's figures, and Exercises 9–11, which ask you to design evaluations.

1. Check the boundaries

List the integers in each interval, then keep the ones that appear in both lists.

First interval and its integers	Second interval and its integers	Shared integers
$[0, 2)$ contains $0, 1$	$[1, 4)$ contains $1, 2, 3$	1
$[1, 4)$ contains $1, 2, 3$	$[4, 7)$ contains $4, 5, 6$	None
$[2, 2)$ is empty	$[0, 5)$ contains $0, 1, 2, 3, 4$	None

The chapter uses half-open intervals. The closing parenthesis in $[a, b)$ means the interval ends just before b. An integer x is inside when $a \le x < b$. This convention lets adjacent nonempty intervals share an endpoint while keeping their integers separate. In the second pair, 4 belongs only to $[4, 7)$. In the third pair, no integer can satisfy $2 \le x < 2$, so the first interval is empty even though the second interval contains 2.

2. Reorder an edit

Suppose an editor needs to fill the body of a function that clips negative integers to zero. The prefix is the function header, and the suffix contains two checks. In file order, the gap is here.

```
def clip_at_zero(n):
    # Missing body

assert clip_at_zero(-3) == 0
assert clip_at_zero(4) == 4
```

A fill-in-the-middle prompt puts the visible code before the marker that tells the model to start generating the body. Here is the prompt with the companion example's generic markers.

```
<fim_prefix>def clip_at_zero(n):
<fim_suffix>
assert clip_at_zero(-3) == 0
assert clip_at_zero(4) == 4
<fim_middle>
```

The model has already seen both assertions when it reaches the middle marker. The first shows that a negative input should return zero. The second shows that a positive input should be preserved, so a body that always returns zero would fail. One body consistent with these clues is `return max(0, n)`, indented inside the function. The editor inserts that line immediately after the function header, before the suffix.

3. Distinguish two expressions

Start with the two cases already in the smoke suite, then add a pair of intervals that touch. The value ℓ is the later start and r is the earlier end.

Intervals	ℓ	r	$\ell \le r$	$\ell < r$	Expected
$[0, 4)$ and $[2, 6)$	2	4	True	True	True
$[0, 2)$ and $[3, 5)$	3	2	False	False	False
$[0, 2)$ and $[2, 5)$	2	2	True	False	False

The smoke suite covers $\ell < r$ and $\ell > r$, where the two expressions agree. Add the third row with an expected result of `False`. The intervals touch at 2, which the first interval excludes. The non-strict comparison $\ell \le r$ accepts equality and therefore reports an overlap where there are zero shared integers. The strict comparison passes the new check.

4. Count successful pairs

Call the passing candidates A and B and the failing candidates C, D, and E. A size-two subset contains two distinct candidates and ignores their order. For example, the pair AB is the same subset as BA. Each candidate can be paired with four others, and this counts each pair twice, so there are $5 \cdot 4/2 = 10$ subsets.

Passing candidates in the pair	Pairs	Count
Two	AB	1
One	AC, AD, AE, BC, BD, BE	6
Zero	CD, CE, DE	3

The first two rows give $1 + 6 = 7$ subsets with at least one pass. With all ten pairs equally likely, the probability of drawing a successful pair is

$$\mathit{pass}@2 = \frac{7}{10} = 0.7.$$

The chapter's formula counts the same event by subtracting the three pairs made entirely of failures, giving $1 - \binom{3}{2} / \binom{5}{2} = 7/10$.

9. Design a repository-context comparison

In a fictional scheduling repository, a room booking from minute 0 to minute 30 requires 10 minutes of cleanup. A second request for minute 35 should be rejected, even though it starts after the original booking ends. A shared utility already defines the occupied interval, including cleanup. The task is to update the availability check to follow that policy.

Choose the function *can_book* in *booking.py*. It should compare a requested booking and an existing one after passing each through *occupied_span* in *time_slots.py*. In this example, the helper converts a nonempty booking to a half-open interval whose excluded right endpoint extends by the cleanup duration. An empty booking has no occupied time. The issue asks for consistent use of the shared utility. Both conditions receive the same issue, function signature, and local file at the same repository revision.

In the **local-only** condition, give the model only that local file and the issue. In the **retrieved-context** condition, also provide the definition and docstring of *occupied_span* retrieved from *time_slots.py* at the same revision. Record the path and the actual lines supplied, so we can check whether retrieval returned the right helper. Give both conditions the same model, number of attempts, output token limit, and decoding settings. Restrict both generators to their supplied inputs. A tool that can read arbitrary repository files would let the local-only condition retrieve the helper too. Use the same input budget with room for the retrieved snippet. If sampling is used, pair the attempt indices and record their seeds. Run proposed edits from both conditions in isolated copies of the same base repository.

Use cases that make the helper's policy observable. In the table, the existing booking is $[0, 30)$ with 10 minutes of cleanup. Each new request is nonempty and has no cleanup of its own.

Held-out request	Expected result	Reason
$[35, 45)$	Reject	The room is occupied until minute 40.
$[40, 50)$	Accept	The occupied interval is $[0, 40)$ and excludes minute 40.
$[50, 60)$	Accept	The requests are separated even with cleanup.

Check the empty case by pairing an existing $[25, 25)$ booking with 10 minutes of cleanup and a $[26, 30)$ request. Accept the request because the empty booking occupies no time. To check cleanup on the request side, pair an existing $[30, 40)$ booking with a $[15, 25)$ request that needs 10 minutes of cleanup. Reject the request because its occupied interval extends to 35. Keep these tests and expected outputs out of the prompts. Report per-case pass rates and the fraction of patches that pass the full suite, with the same selection rule in each condition. A gain on the cleanup or empty-booking cases would show where the retrieved definition helped. Also retain ordinary cases, such as the separated pair, to check for regressions.

10. Check for regressions

The issue describes two bookings with one end equal to the other start. Use the chapter's half-open contract. A booking $[0, 2)$ occupies 0 and 1, and a booking $[2, 4)$ occupies 2 and 3. These sets do not share an integer. If the existing function uses the non-strict comparison $\ell \leq r$, it returns true when the later start ℓ equals the earlier end r.

Run the same three tests on the buggy version and the patch. The expected values express the contract in both runs.

Purpose	Bookings	Expected	Buggy comparison
Reproduce the reported bug	$[0, 2)$ and $[2, 4)$	False	$2 \leq 2$, true
Preserve a genuine overlap	$[0, 3)$ and $[2, 4)$	True	$2 \leq 3$, true
Preserve separation	$[0, 2)$ and $[3, 4)$	False	$3 \leq 2$, false

The first test fails before the fix and must pass afterward. The other two pass before and must still pass. Returning false for every pair would satisfy the first test and break the overlap case. A strict comparison $\ell < r$ makes all three examples agree with the contract.

Give the model the issue report, a description of the observed symptom, and the existing function and its contract. For example, the report can say that a room must be bookable when one slot starts exactly when another ends. Keep the evaluator's concrete inputs, test assertions, and any reference patch separate. If the original issue includes a public reproducer, the model can see it, and the evaluator should keep an additional adjacent pair private. Run the rest of the repository's tests as well. Together these checks measure whether the patch fixes the reported boundary and preserves working behavior.

11. Audit a benchmark split

Suppose two public issues describe the same interval bug in an upstream project and a fork. The issue titles differ, but the patch in both changes a non-strict comparison to a strict one in nearly identical code. Putting one task in training and the other in test can reward recall of the patch. Splitting by issue number or date alone can leave that leak.

Create groups of repositories that share a lineage, including forks and renamed mirrors. Record each task's issue date, base commit, patch date, and the origin of its tests. Choose dates and repository groups before running the evaluation. For example, train on earlier tasks from one set of repository families, validate on later tasks from a second set, and test on the latest time window from a third set. Freeze both the cutoff dates and the groups. Keep a family in one partition even if that means discarding its tasks outside the partition's time window. Report the number of tasks and repository families that remain in each split.

Look for duplicates at more than one level. Use exact hashes to catch copies, then compare normalized issue text, edited functions, tests, and patch hunks to flag candidates for review. For example, renaming variables may change the raw diff while leaving the same syntax and test behavior. Inspect flagged pairs for the same underlying issue and fix, group duplicates before splitting, and keep each cluster together or drop copies that cross a split. Save the deduplication criteria and report how many tasks were excluded.

For each evaluation task, expose only a repository snapshot from before the fix and the issue text available at the defined task time. A retrieval index should use that snapshot, including its documentation. Exclude solution pull requests, later versions of the issue with pasted fixes, commit history that reaches the patch, and held-out tests from generation and retrieval. Restrict generation tools to the frozen inputs so they cannot browse later history or query the live network. The scoring environment applies the candidate edit to the base revision and runs the tests separately. A spot check can search the supplied materials for exact patch lines and test fixtures. Behavior on the held-out tests determines credit.

These controls address overlap in the benchmark and in its supplied context. Public issue text or code could still have appeared in a model's pretraining data. Record the model version, any documented training cutoff, and when the artifacts became public. Report results by date or exposure category where possible and state that a cutoff or lack of a found duplicate cannot establish what a model saw. List any confirmed overlap, suspected overlap, and unknown cases separately so readers can tell which conclusions depend on uncertain exposure.